

ITSS SOFTWARE DEVELOPMENT/ SOFTWARE DESIGN AND CONSTRUCTION

Lab 10: Unit Testing

Lecturer: NGUYEN Thi Thu Trang, trangntt@soict.hust.edu.vn

1. SUBMISSION GUIDELINE

To submit this lab, you and your teammate have to push all the work to the group GitHub repository, following the naming convention `MSTeamsName-TeamNumber` (e.g., TKXDPM.VP.20251-01).

For this lab, you are required to check out a branch named `release/lab10`, contains all tasks related to unit testing.

The deadline for submission will be updated by the lecturer or the teaching assistant. Therefore, you need to pay attention to avoid late submissions.

2. OVERVIEW

In this lab, we will get familiar with the software construction process and try ourselves with unit testing (see **3.1**) for the Case Study AIMS.

In this course, you are free to choose the framework and programming language (as long as they support object-oriented programming) to develop the project. Therefore, the implementation of unit tests will largely depend on which framework and programming language are used.

However, regardless of the choice, we will still use the test-driven development (TDD) technique to develop the project (see **3.2**), along with the black-box testing technique to create test cases (a technique that relies only on the software's input and output requirements without considering the internal source code).

In this lab, due to some constraints, we only introduce JUnit 5 (see **3.3** and **3.4**) – a widely used unit testing framework in the Java programming language and its related frameworks (such as Spring Boot). You can refer to a sample project¹ if need be. **If you choose a different framework and programming language, please refer to their documentation for further information** (for example, PHPUnit is widely used in PHP programming language and Laravel framework).

¹ <https://github.com/trangntt-for-student/AIMS>

3. UNIT TESTING

3.1. OVERALL

Testing plays a crucial role in software development and helps to determine whether a property of the program holds or not.

A well-tested program will have tests for every individual module (where a module is a method or a class) that it contains. A test that tests an individual module, in isolation if possible, is called a unit test. Testing modules in isolation leads to much easier debugging. When a unit test for a module fails, you can be more confident that the bug is found in that module, rather than anywhere in the program².

Thus, unit testing is neither suitable for testing complicated user interface nor the interaction among great modules/subsystems.

3.2. TEST-DRIVEN DEVELOPMENT (TDD)³

Test-driven development (TDD), which is rooted in extreme programming, is all about satisfying your team that the code works as expected for a behavior or use case. Instead of aiming for the optimum solution in the first pass, the code and tests are iteratively built together one use case at a time. Development teams use TDD as part of many coding disciplines to ensure test coverage, improve code quality, set the groundwork for their delivery pipeline, and support continuous delivery.

In the article *TDD - What it is and what it is not*⁴, Andrea Koutifaris has described this cycle as three phases:

- **Red phase:** You write an automated test for a behavior that you're about to implement. Based on the user requirement, you decide how you can write a test that uses a piece of code as if it were implemented. This is a good opportunity to think about the externals of the code, without getting distracted by actually filling in the implementation. Think about what the interface should look like? What behaviors should a caller of that interface expect? Is the interface clean and consumable?
- **Green phase:** You write production code, but only enough production code to pass the test. You don't write algorithms, and you don't think about performance. You can duplicate code and even violate best practices. By

² http://web.mit.edu/6.031/www/sp17/classes/03-testing/#automated_unit_testing_with_junit

³ https://www.ibm.com/garage/method/practices/code/practice_test_driven_development/

⁴ <https://medium.freecodecamp.org/test-driven-development-what-it-is-and-what-it-is-not-41fa6bca02a2>

addressing the simplest tasks, your code is less prone to errors and you avoid winding up with a mix of code: some tested (your minimalist functions) and some untested (other parts that are needed later).

- **Refactor phase:** You change the code so that it becomes better. At a minimum, you remove code duplication. Removing duplication generally leads to abstraction. Your specific code become more general. The unit tests provide a safety net which supports the refactoring, because they make sure that the behavior stays the same and nothing breaks. In general, tests should not need to be changed during the refactor stage.

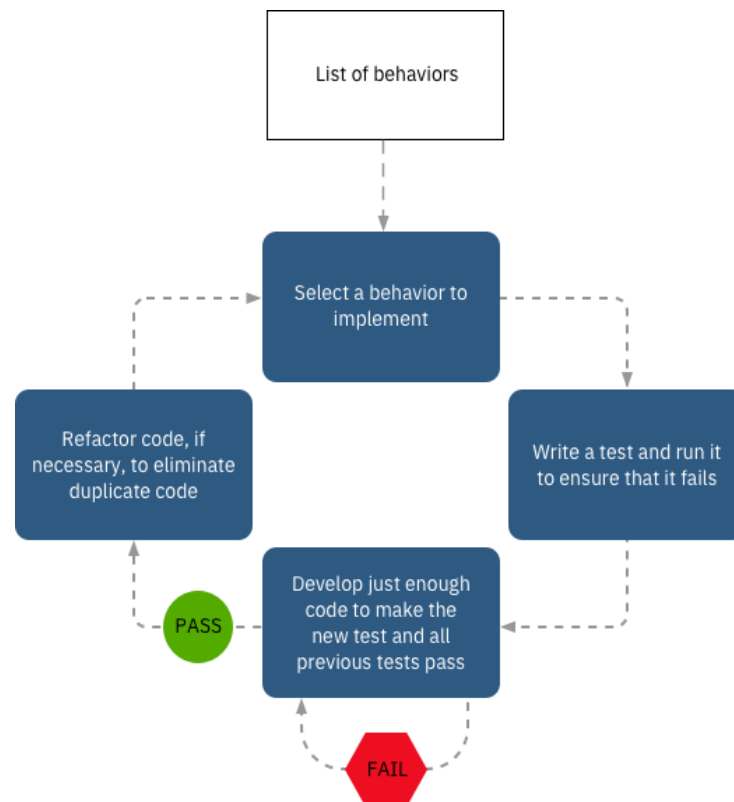


Figure 1- Red/Green/Refactor cycle

TDD drives the code development, and every line of code has an associated test case, so unit testing is integrated into the practice. Unit testing is repeatedly done on the code until each unit functions per the requirements, eliminating the need for you to write more unit test cases.

You can get started with TDD by following these steps:

Step 1: Think about the behaviors that your implementation requires, then select a behavior to implement.

Step 2: Write a test that validates the behavior. The test case must fail.

Step 3: Add only enough code to make the new test case and all previous test cases pass.

Step 4: Refactor the code to eliminate duplicate code, if necessary.

Step 5: Select the next requirement and repeat steps 1 - 4.

3.3. UNIT TESTING WITH JUNIT5

3.3.1. Automated Unit Testing with JUnit⁵

JUnit is a widely-adopted Java unit testing library, and we will use it heavily in this course. A JUnit unit test is written as a method preceded by the annotation `@Test`. A unit test method typically contains one or more calls to the module being tested and then checks the results using assertion methods like `assertEquals`, `assertTrue`, and `assertFalse`.

For example, the tests we chose for `Math.max()` above might look like this when implemented for JUnit:

```
@Test
public void testALessThanB() {
    assertEquals(2, Math.max(1, 2));
}

@Test
public void testBothEqual() {
    assertEquals(9, Math.max(9, 9));
}

@Test
public void testAGreaterThanB() {
    assertEquals(-5, Math.max(-5, -6));
}
```

Note that the order of the parameters to `assertEquals` is important. The first parameter should be the expected result, usually a constant, that the test wants to see. The second parameter is the actual result, what the code actually does. If you switch them around, then JUnit will produce a confusing error message when the test fails. All the assertions supported by JUnit follow this order consistently: expected first, actual second.

⁵ http://web.mit.edu/6.031/www/sp17/classes/03-testing/#automated_unit_testing_with_junit

If an assertion in a test method fails, then that test method returns immediately, and JUnit records a failure for that test. A test class can contain any number of `@Test` methods, which are run independently when you run the test class with JUnit. Even if one test method fails, the others will still be run.

3.3.2. Documenting Your Testing Strategy⁶

Let consider a function that reverses the end of a string.

```
/**
 * Reverses the end of a string.
 *
 * For example:
 *   reverseEnd("Hello, world", 5)
 *   returns "Hellodlrow ,"
 *
 * With start == 0, reverses the entire text.
 * With start == text.length(), reverses nothing.
 *
 * @param text    non-null String that will have
 *                 its end reversed
 * @param start    the index at which the
 *                 remainder of the input is
 *                 reversed, requires 0 <=
 *                 start <= text.length()
 * @return input   text with the substring from
 *                 start to the end of the string
 *                 reversed
 */
static String reverseEnd(String text, int start)
```

For example, at the top of the class, we can document the testing strategy we worked on in the partitioning exercises above. The strategy also addresses some boundary values we did not consider before.

⁶ [http://web.mit.edu/6.031/www/sp17/classes/03-testing/#automated unit testing with junit](http://web.mit.edu/6.031/www/sp17/classes/03-testing/#automated%20unit%20testing%20with%20junit)

```

/*
 * Testing strategy
 *
 * Partition the inputs as follows:
 * text.length(): 0, 1, > 1
 * start:          0, 1, 1 < start < text.length(),
 *                  text.length() - 1, text.length()
 * text.length()-start: 0, 1, even > 1, odd > 1
 *
 * Include even- and odd-length reversals because
 * only odd has a middle element that doesn't move.
 *
 * Exhaustive Cartesian coverage of partitions.
 */

```

Each test method should also need a comment above it saying how its test case was chosen, i.e. which parts of the partitions it covers:

```

// covers test.length() = 0,
//          start = 0 = text.length(),
//          text.length()-start = 0
@Test
public void testEmpty() {
    assertEquals("", reverseEnd("", 0));
}

```

3.3.3. Using annotations⁷

JUnit 5	Description
@Test	The annotated method is a test method.
@BeforeAll	The annotated (static) method will be executed once before any @Test method in the current class.

⁷ <https://developer.ibm.com/tutorials/j-introducing-junit5-part1-jupiter-api/>

JUnit 5	Description
@BeforeEach	The annotated method will be executed before each @Test method in the current class.
@AfterEach	The annotated method will be executed after each @Test method in the current class.
@AfterAll	The annotated (static) method will be executed once after all @Test methods in the current class.
@Disabled	The annotated method will not be executed (it will be skipped), but reported as such.

Figure 2-Important Annotations in JUNIT5

3.4. UNIT TESTING AS A PART OF TDD

3.4.1. Test case Design

In this part, we would design test cases for the method `validateDeliveryInfo()` in class `PlaceOrderController`. This is only an example to demonstrate how to create test case. Since we do not have the source code yet, the **black-box testing** method will be used in this case.

The method `validateDeliveryInfo()` requires customer's name, phone number, and address as the input. Therefore, we can divide the method `validateDeliveryInfo()` into 3 three submethods: `validateAdress()`, `validateName()`, and `validatePhoneNumber()`. Initially, all these methods are empty. Remember to write down in the report how you can have such test cases with which techniques.

#	Parameter	Conditions
1	Name	Only letters (a-z and A-Z) are allowed Maximum of 30 characters Must not null
2	Phone number	Only numbers (0-9) are allowed Must have 10 digits, start with 0 May include separators such as dots (.), hyphens (-) or slashes (/), but only one type of separator can be used in each phone number, interleaved with the digits.

#	Parameter	Conditions
3	Address	Only letters (a-z and A-Z) or digits (0-9) or slashes are allowed Maximum of 100 characters Must not null

3.4.2. Test case Writing

Based on SRS and SDD, we design unit tests and develop the method `validateDeliveryInfo()` by applying TDD.

The screenshot shows the Eclipse IDE with the file `PlaceOrderController.java` open. The Package Explorer on the left shows the project structure with `PlaceOrderController.java` selected. The main editor displays the following Java code:

```

66  * @throws InterruptedException
67  * @throws IOException
68  */
69  public void processDeliveryInfo(HashMap info) throws InterruptedException, IOException{
70      LOGGER.info("Process Delivery Info");
71      LOGGER.info(info.toString());
72      validateDeliveryInfo(info);
73  }
74
75  /**
76   * The method validates the info
77   * @param info
78   * @throws InterruptedException
79   * @throws IOException
80   */
81  public void validateDeliveryInfo(HashMap<String, String> info) throws InterruptedException, IOException{
82  }
83
84  public boolean validatePhoneNumber(String phoneNumber) {
85      return false;
86  }
87
88
89  public boolean validateName(String name) {
90      return false;
91  }
92
93  public boolean validateAddress(String address) {
94      return false;
95  }

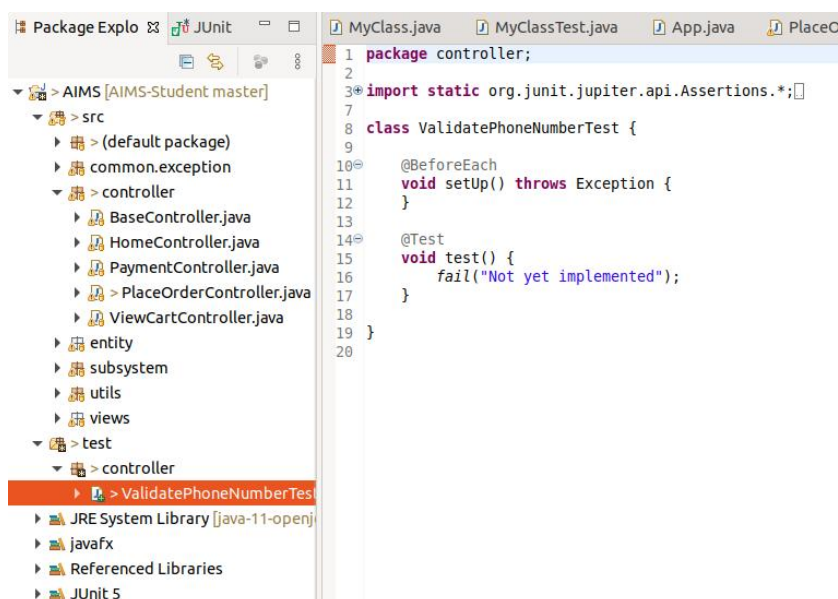
```

3.4.3. Creating unit test class

To illustrate, we create a new class for the testing the method `validatePhoneNumber()`.

You may be asked to import JUNIT5 library as follows.

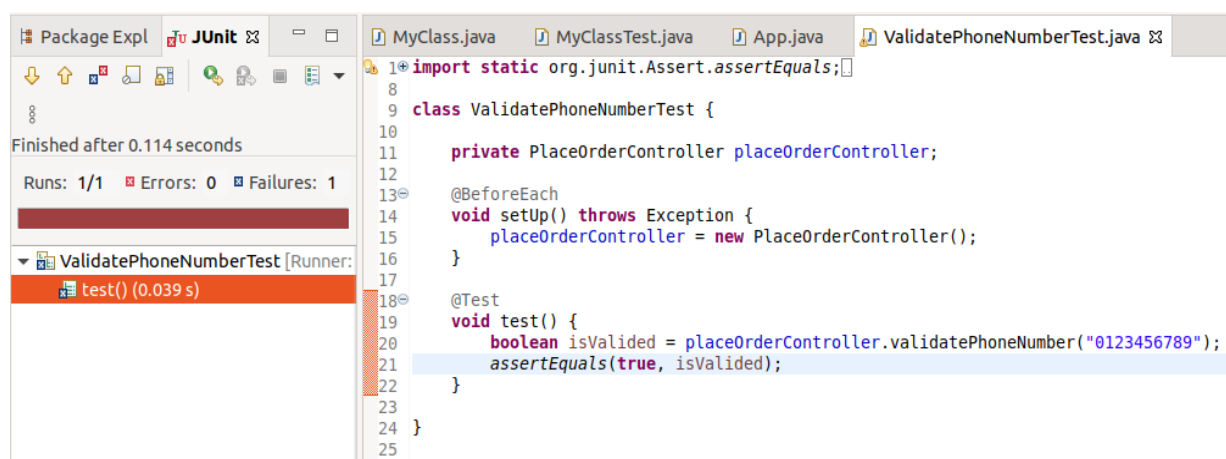
The result is shown as follows.



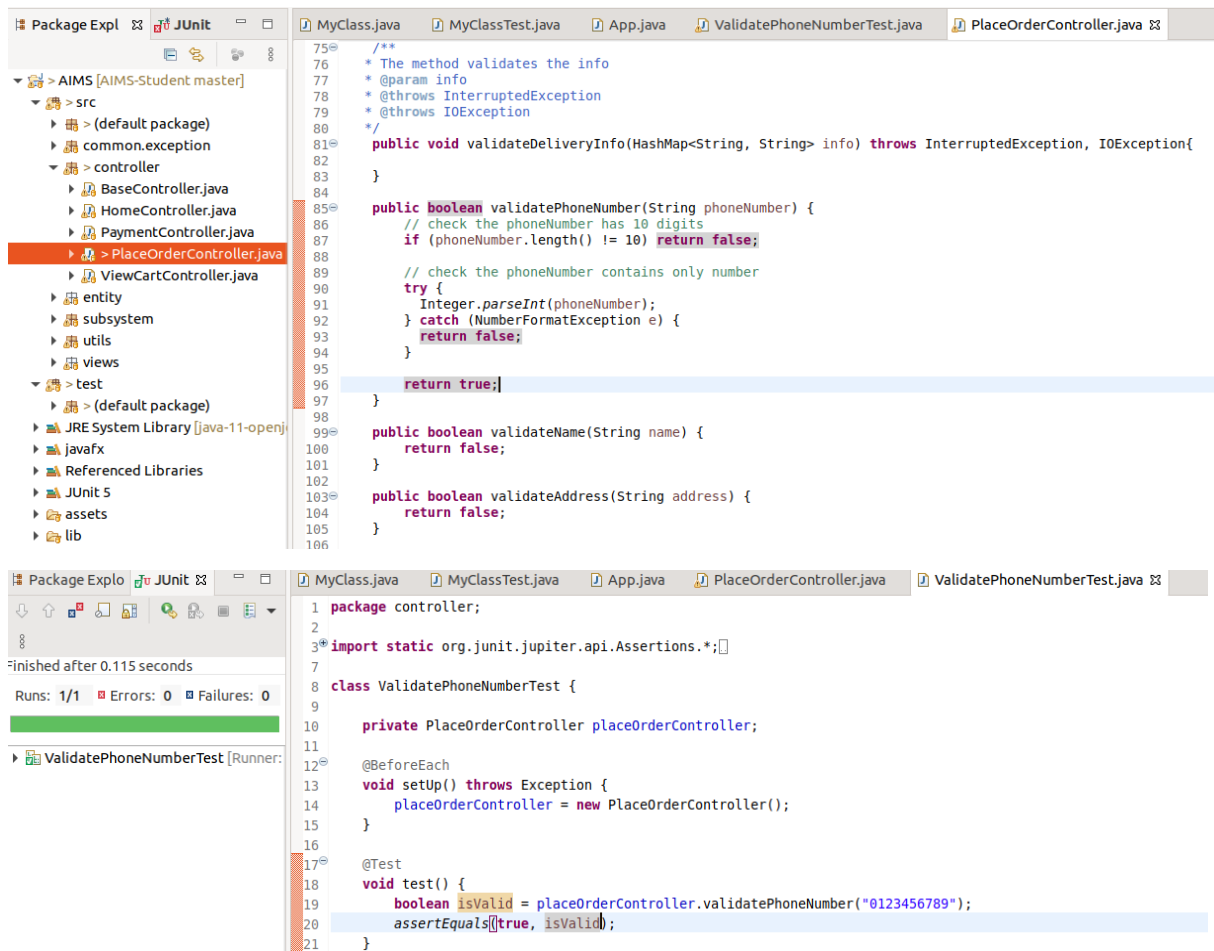
Obviously, if we run the test now, the test will be failed since we have not implemented anything yet.

3.4.4. Development with TDD

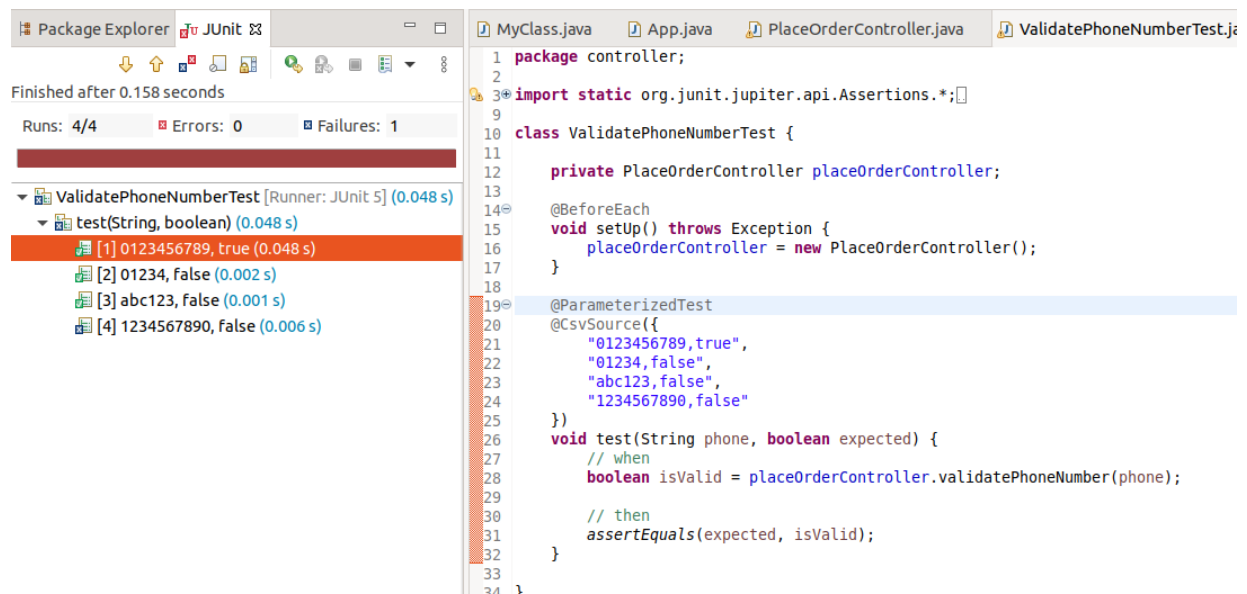
Now, if we create an object of `PlaceOrderController` class and then try testing the method `validatePhoneNumber()` with a valid phone number, e.g., 0123456789, the test will be failed since the method return false by default.



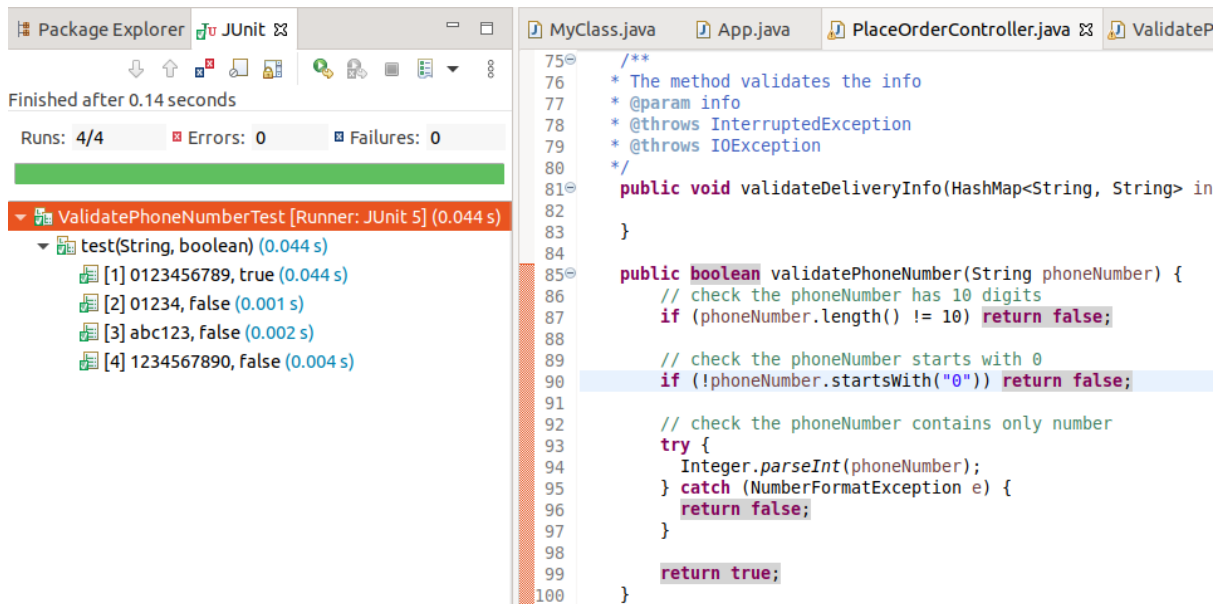
Thus, we need to go back to `PlaceOrderController` class to implement `validatePhoneNumber()` method. After that, the test case for this method should pass.



We can put a list of input-expected output pairs to test at once by using annotations `@Parameterized` and `@CsvSource`. After that, run the test again.



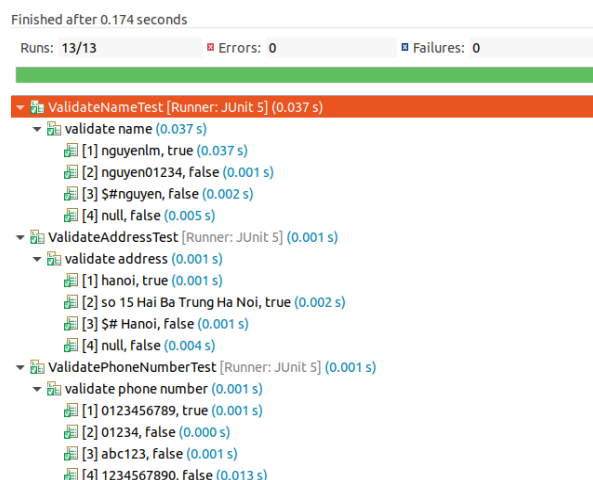
You can see that there is a failed test case: the one with the input-output pair "1234567890, false". We need to go back to the method and add the validation codes for the phone number which must start with 0. At last, we have all passed test cases.



3.4.5. Working with test suite

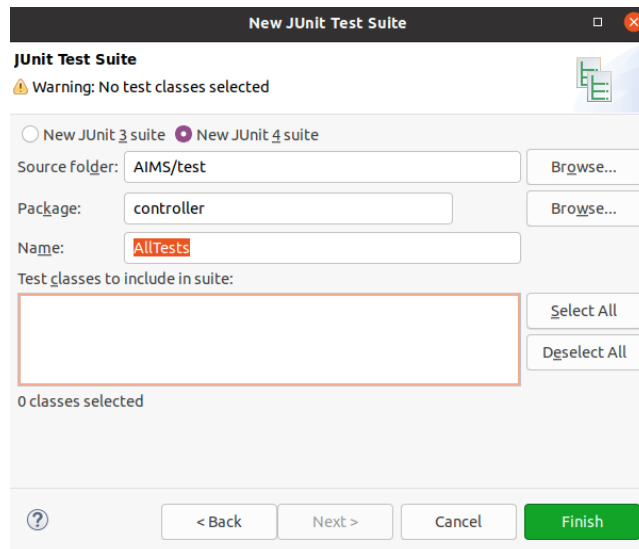
You are asked to implement 2 methods `validateAdress()` and `validateName()` by yourself.

After you finish the task, we would have 3 testing classes. Eclipse can run all the three test cases at once. The result is shown as follows.

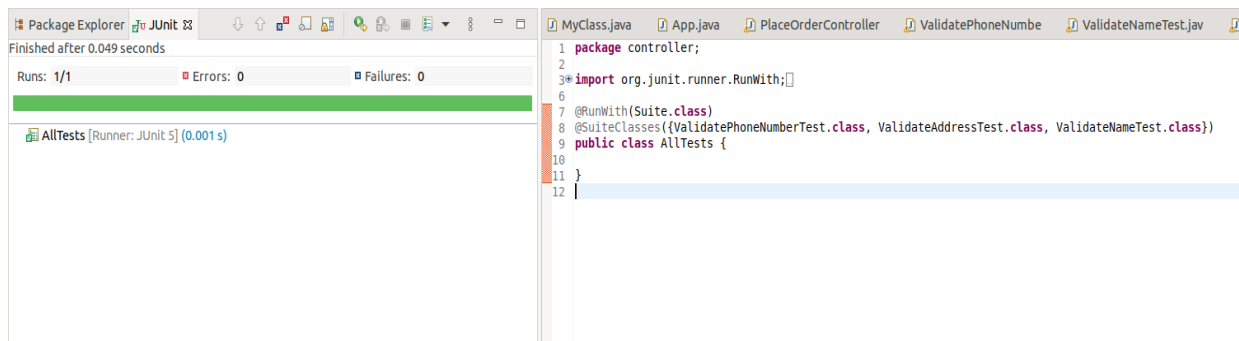


In another hand, these three test cases could make a test suite. A test suite is a collection of test cases related to the same test work.

Right click on the project -> New -> Test Suite



Add all classes that we need to test in the test suite into class `AllTest` and then run.



3.5. HOMEWORK

Now, you will carry out unit testing using the TDD technique described above, along with the black-box testing method. Each member of the group is required to select AT LEAST ONE MAIN unit in the project to perform unit testing. To create the test cases, you will refer to the template “**(TEMPLATE-EN) TestPlan-UnitTest**” provided by the lecturer.

To ensure easy identification between different teams and document versions, please follow the naming convention:

[TEAM-NO]TestPlan-UnitTest-[VerXx]

- [TEAM-NO]: your team number (e.g., TEAM-01)

- [Ver-Xx]: the current version of the document (e.g., Ver-1.0)

In the root directory, you are required to create a directory named `UnitTesting/` to submit your work. You also need to submit all the images (in PNG or JPG format) which demonstrate that you have passed all the designed test cases for the units you are in charge of. Don't forget to name the files according to the corresponding unit.

```
UnitTesting/  
└─ TestPlan  
    └─ [TEAM-NO]TestPlan-UnitTest[VerXx].xlsx  
    └─ [TEAM-NO]TestPlan-UnitTest[VerXx].pdf  
└─ Images  
    └─ UnitA.png  
    └─ ... (put your files here)
```

For the source code implementing the tests, you will follow the project structure of the framework and programming language you have chosen. For example, for groups that choose Java and its related frameworks (such as Spring Boot), your source code may be organized as follows:

```
src/  
└─ main/  
    └─ java/  
        └─ com.hust.aims/  
            └─ ... (your code here)  
    └─ resources/  
└─ test/  
    └─ java/  
        └─ com.hust.aims/  
            └─ ... (your code here)
```